In Class Lab 5d

Auto Increment, Outer Joins and Your First Function | CIS 4720 Database Management and SQL

Points	Time	Work in	Turn in
10	About 90 minutes	student_practice.sql	Your GitHub repository URL

The due date is listed on Canvas LMS.

Four things to understand before you start

1. AUTO_INCREMENT

Every table needs a primary key, and so far you have been typing the key values in yourself. That works for four rows and falls apart for four thousand, because somebody has to remember which numbers are taken.

AUTO_INCREMENT hands that job to the server. Put it on an integer column and the server keeps a counter for that table. Every time you insert a row without supplying a value, it uses the counter and then adds one. You never pick an id again, and two rows can never collide.

What you need to know	Why it matters
You get the next number, not the lowest free one	Delete row 6 and the next insert is still 7. The counter only ever goes up
Gaps are normal and are not a problem	A failed insert can consume a number. Ids are labels, not a count of your rows
Insert NULL and the server fills it in	That is what Step 2 is doing. NULL here means you decide, not this is empty
Only one AUTO_INCREMENT column per table	And it has to be indexed, which a primary key already is

2. LAST_INSERT_ID()

If the server picked the id, how do you find out what it picked? You ask. **LAST_INSERT_ID()** gives back the AUTO_INCREMENT value generated by your most recent insert. In real code this is how you add an order and then add the order's line items, which need the order's id to point at.

> **Two things about it that surprise people**
>
> **After a multiple row insert it returns the FIRST id of the batch, not the last.** Insert four rows in one statement and it comes back as 1, not 4. That is correct and deliberate: with the first id and the number of rows, you can work out all of them.
>
> **It is per connection.** It remembers what YOUR session did, which is what makes it safe on a busy server where other people are inserting at the same time. The flip side is that a brand new connection has nothing to remember and returns **0**. If you get 0, your tool opened a new connection, not a wrong answer.

3. What a function is in MySQL

A function is a piece of SQL you save on the server under a name, which takes arguments, runs some logic, and hands one value back. Once it exists you can call it anywhere you would use a column or a number, including

inside a SELECT or an UPDATE.

The point is that the logic lives in one place. The rule for working out a grade average is written once, on the server, and every query that needs it calls the same function. Change the rule and nothing else has to be found and edited.

A view and a function are cousins. A view stores a SELECT and hands back rows. A function stores logic, takes an input, and hands back a single value.

4. The clauses you will see in Step 8

Clause	What it does
DELIMITER	Not SQL at all. It is an instruction to the mysql command prompt program, which chops your typing into statements at every semicolon before sending them on. A function body is full of semicolons, so you tell the command prompt to chop at $$ instead, write the function, then set it back. The server never sees these lines. Send one to the server and it has no idea what you mean
CREATE FUNCTION	Names the function and lists the arguments it takes, with a data type on each
RETURNS	The data type of the single value that comes back. One value, one type, always
DETERMINISTIC	Your promise that the same input always gives the same output. The server trusts it, so declare it only when it is true
READS SQL DATA	Tells the server this function queries tables but does not change them
DECLARE	Creates a variable to hold a value while the function is working. It exists only inside the function and disappears when it finishes
BEGIN and END	Wrap the body. Everything between them is the function's logic

Where you run Steps 8 and 10, and why it matters

Because DELIMITER is a command prompt instruction and not SQL, **running these two steps inside VS Code fails**. SQLTools passes whatever you give it straight to the server, the server has never heard of DELIMITER, and you get:

```
ERROR 1064 ... near 'DELIMITER $$' at line 1
```

So do Steps 8 and 10 at the **command prompt**, in the window you left open in setup. That is where DELIMITER belongs and it works there. Each of those steps shows you exactly what to do.

A fair question about DETERMINISTIC

The function in Step 8 is declared DETERMINISTIC and it also reads a table. Strictly those pull against each other: if someone changes a score, the same student id gives a different answer, so it is not really deterministic.

MySQL does not check, and the declaration is what lets the function be created at all when binary logging is switched on. It is fine here. On a production server you would write NOT DETERMINISTIC for a function that reads tables, and it is worth knowing the difference now rather than later.

Part 1. Connect and set up

There is a chicken and egg problem at the start of this lab. SQLTools will not connect to a schema until one exists, and the schema you want does not exist yet. The way round it is to connect to a schema that is always there, and use that connection to create your own.

Step 1 of setup. Connect to a schema that already exists

In VS Code, open the SQLTools panel, choose **Add New Connection** and pick **MySQL**. Fill in server **localhost**, port **3306**, username **root** and your password, and for the database use one of these:

Database	When to use it
information_schema	Always there on every MySQL server, so this one cannot fail. Use it unless your instructor says otherwise
sakila	A sample database. Use it only if it is installed on your PC. If it is not in the list, use information_schema

Test the connection, then save it. A green tick means you are in.

Step 2 of setup. Make your working file

Make a folder on OneDrive called **CIS4720_Lab5d**, through the OneDrive app rather than the browser, and open that folder in VS Code with **File**, then **Open Folder**. Download **student_practice.sql** from Canvas into it.

The file already has the two setup statements written in, plus a numbered heading for every step. Run those two first, one at a time:

```
show databases;


create database if not exists cis_students_db;
```

The first lists every schema on the server. The second makes yours. Run **show databases;** again afterwards and **cis_students_db** should now be in the list.

Step 3 of setup. Point yourself at the new schema

This is the step people skip, and then spend twenty minutes wondering why their tables are not where they put them. You are still connected to information_schema. Anything you create right now tries to go there.

The file has this line in it, which switches the session over:

```
use cis_students_db;
```

Run it. Then, to be safe, also add a second SQLTools connection with **Database** set to **cis_students_db** and use that one from Step 1 onwards. Some tools open a fresh connection for every statement, and a fresh connection forgets the USE. Having the connection itself point at the right schema means it cannot go wrong.

> **How to tell which schema you are actually in**
>
> Run **SELECT DATABASE();** at any point. It names the schema your statements are landing in right now. If it says information_schema or NULL when you are expecting cis_students_db, stop and fix that before writing anything else.

Part 2. Build the tables

You write the SQL for Steps 1 through 7. The specification and the expected result are here, and your textbook, the course lab guide and your earlier lab files are all fair game for the syntax. Put a comment above every statement saying what it does.

Step 1. Create the students table

Create a table named **students** with these four columns, in this order:

Column	Type	Rules
student_id	INT	AUTO_INCREMENT, and the primary key
first_name	VARCHAR(40)	Never empty
last_name	VARCHAR(40)	Never empty
gpa	FLOAT(4,2)	May be left empty. Four digits with two after the point

Then describe the table. You should see this, and the **Extra** column on the right is the part to look at:

Field	Type	Null	Key	Default	Extra
student_id	int	NO	PRI	NULL	auto_increment
first_name	varchar(40)	NO		NULL	
last_name	varchar(40)	NO		NULL	
gpa	float(4,2)	YES		NULL	

Step 2. Insert four rows, letting the server pick the ids

Insert these four students in a single statement. Supply **NULL** for student_id and **NULL** for gpa. The NULL in student_id is what tells AUTO_INCREMENT to do its job. The NULL in gpa is a real empty value, which you fill in at the end of the lab.

first_name	last_name
Jenna	Wilson
Robert	Reese
Maria Elena	Jimenez
Inder	Singh

Then run these two:

```
select last_insert_id();

select * from students;
```

The four rows come back with ids **1, 2, 3 and 4**, which you never typed. And last_insert_id() returns **1**.

> **Why 1 and not 4**
>
> You inserted four rows and the last one got id 4, so 1 looks wrong. It is not. After a multiple row insert, LAST_INSERT_ID() reports the id of the **first** row in the batch. From that number and the row count you can work out every id the batch used, which is exactly what application code needs.
>
> If you get **0** instead, your tool ran that statement on a new connection. LAST_INSERT_ID() only remembers what the current session did. Run the insert and the select together as one block and you will see the real number.

Step 3. Insert two more rows, naming the columns first

This time list the column names you are supplying, **(first_name, last_name)**, and then the values. Naming the columns means you can leave student_id and gpa out altogether rather than passing NULL for them, which is

the form you will use most often in practice.

first_name	last_name
Rita	Nunez
James	Toor

Then run last_insert_id() and select the table again. Six rows now, and last_insert_id() returns **5**, which is Rita, the first row of this batch.

Step 4. Delete James Toor

Delete the row for James Toor, then select the table again. Five rows left, with ids 1 through 5.

> **Write the WHERE clause first**
>
> A DELETE with no WHERE empties the whole table, with no confirmation and no undo. Write it as a SELECT first, check that exactly one row comes back, and only then change the word SELECT to DELETE.

Now look at what the counter did. Id 6 is gone and nothing will ever reuse it. The next row you insert gets **7**. That is AUTO_INCREMENT working as designed: the counter only moves forward, so an id that existed once can never point at something else later.

Step 5. Create the test_scores table

Create a table named **test_scores** with these four columns:

Column	Type	Rules
test_score_id	INT	AUTO_INCREMENT, and the primary key
test_name	VARCHAR(20)	Never empty
student_id	INT	Never empty. A foreign key, see below
score	FLOAT	May be left empty

Add a **foreign key** on student_id pointing at **student_id in the students table**. That is the rule saying a score can only belong to a student who really exists.

> **Point the foreign key at the right column name**
>
> The column in students is called **student_id**, not id. Write **references students(id)** and the table is refused with **ERROR 1005, errno 150, foreign key constraint is incorrectly formed**, which does not mention the column name and is why it confuses people.
>
> A foreign key has to name a real column, and that column has to be a key in the other table. Here it is the primary key, which is exactly what you want.

Step 6. Insert the ten test score rows

Insert these ten rows in one statement, using NULL for test_score_id so AUTO_INCREMENT fills it in:

test_name	student_id	score	test_name	student_id	score
quiz1	1	90	quiz2	4	91
quiz1	2	85	quiz3	2	95

test_name	student_id	score	test_name	student_id	score
quiz1	3	95	quiz3	3	94
quiz2	1	80	quiz3	4	74
quiz2	2	72			
quiz2	3	88			

Then select everything from test_scores. Ten rows. Notice that student 5, Rita, has no scores at all. That is deliberate and Step 7 depends on it.

Step 7. Save a LEFT OUTER JOIN as a view

A plain join returns only the rows that match on both sides. Join students to test_scores that way and Rita disappears, because she has no scores to match. The report would quietly claim you have four students.

A **LEFT OUTER JOIN** keeps every row from the table on the left whether or not it has a match on the right, and puts NULL in the right hand columns where there was nothing to join to. Nobody is dropped.

This time you are not just running the join, you are storing it. Create a view named **student_scores** built on that join, returning these four columns:

Column in the view	Where it comes from
student_id	students
student_name	Calculated. The first name and the last name joined together with a single space between them, with student_name as its heading
test_name	test_scores
score	test_scores

Join **students** on the left to **test_scores** on the right, matching **student_id** in students against **student_id** in test_scores.

> **Joining two columns of text into one**
>
> student_name does not exist in either table. You build it in the SELECT from the two name columns, with a space between them, and give the result a heading. That is a function you met in Chapter 4, and the lab guide shows it.
>
> Getting **JennaWilson** with no space means you joined the two columns and forgot the space in the middle. The space is a third thing to join, not something the function adds for you.

Then query the view twice. First everything in it:

```
select * from student_scores;
```

Eleven rows, with the names already joined up:

student_id	student_name	test_name	score
1	Jenna Wilson	quiz1	90
1	Jenna Wilson	quiz2	80
2	Robert Reese	quiz1	85
...	...	...	...

student_id	student_name	test_name	score
4	Inder Singh	quiz3	74
5	Rita Nunez	NULL	NULL

Ten rows have scores and the eleventh is Rita with NULL in both columns. That last row is the entire point of the outer join. Swap it for a plain join and run it again to watch her vanish.

Now ask the view about one student only:

```
select * from student_scores where student_id = 3;
```

Three rows, all of them Maria Elena Jimenez:

student_id	student_name	test_name	score
3	Maria Elena Jimenez	quiz1	95
3	Maria Elena Jimenez	quiz2	88
3	Maria Elena Jimenez	quiz3	94

Notice what you did not have to write. The join, the CONCAT and the outer join are all inside the view now, so filtering it takes a plain WHERE clause on a view that behaves exactly like a table.

Part 3. Write your first function

Both statements in this part are printed for you, because you have not written a function before. Copy them, but read the comments as you go. You will be asked about these clauses later.

Step 8. Create the calculate_gpa function

This function takes a student id and gives back that student's average score. Every line is commented.

> **Run this at the command prompt, not in VS Code**
>
> 1. Paste the whole block below into **student_practice.sql** under the Step 8 heading, so it is in the file you submit. Do not run it there.
>
> 2. Switch to the command prompt window you left open in setup. If you closed it, open one and run **mysql -u root -p**, then **use cis_students_db;**
>
> 3. Copy the whole block and paste it into that window, then press Enter.
>
> The DELIMITER lines are instructions to that program, so this is the one place they work. Paste them into VS Code instead and you get **error 1064 near 'DELIMITER $$'**, because the server was handed a word that is not SQL.

```
-- stop treating ; as the end of a statement, and use $$ instead,
-- so the semicolons inside the function do not cut it short
DELIMITER $$

-- name the function and say it takes one argument, a student id
CREATE FUNCTION calculate_gpa(p_student_id INT)

-- the single value it hands back is a number with 2 decimal places
RETURNS DECIMAL(4,2)

-- promise that the same id always produces the same answer
DETERMINISTIC

-- it queries a table but never changes one
READS SQL DATA

-- the body starts here
BEGIN
    -- a variable to hold the answer while we work it out
    DECLARE v_gpa DECIMAL(4,2);

    -- average that student's scores, rounded to 2 decimal places
    SELECT ROUND(AVG(score), 2)
    -- and put the result into the variable instead of returning it
    INTO v_gpa
    FROM test_scores
    -- only this student's rows
    WHERE student_id = p_student_id
      -- and ignore any row with no score recorded
      AND score IS NOT NULL;

    -- hand back the answer. a student with no scores averages to
    -- NULL, so IFNULL turns that into 0.00 rather than nothing
    RETURN IFNULL(v_gpa, 0.00);
END$$

-- put the delimiter back to a semicolon for everything that follows
DELIMITER ;
```

If you would rather stay in VS Code

There is a version that works there. Leave out the three DELIMITER lines completely, keep everything from CREATE FUNCTION down to END, drop the $$ after END, and run that as a single statement.

The server does not mind the semicolons inside BEGIN and END. It only ever needed one statement at a time, and the whole function is one statement. DELIMITER exists to stop the command prompt program chopping it up, so without that program you do not need it.

The catch is that your editor may chop it up instead. If you get an error on the DECLARE line, that is what happened, and the command prompt is the reliable way through.

Then confirm the server has it, and read back what it stored:

```
show create function calculate_gpa;
```

Step 9. Run the function on every student

Two statements:

```
UPDATE students
SET gpa = calculate_gpa(student_id);

select * from students;
```

What that UPDATE is actually doing

It walks every row in students. For each one it calls calculate_gpa, passing in that row's own student_id, and writes whatever comes back into that row's gpa column.

So the function runs once per student, each time with a different argument, and each time it goes away and averages that student's rows in test_scores. One statement, five separate calculations, and no loop written anywhere.

There is no WHERE clause, and here that is correct: every student is meant to be updated. It is the one case where leaving WHERE off is deliberate rather than a mistake.

The select afterwards should give you exactly this:

student_id	first_name	last_name	gpa
1	Jenna	Wilson	85.00
2	Robert	Reese	84.00
3	Maria Elena	Jimenez	92.33
4	Inder	Singh	82.50
5	Rita	Nunez	0.00

Check Rita's row. She has no scores, so the average came back as NULL and the **IFNULL** in the function turned it into 0.00. Without that line her gpa would still be empty, and the function would have looked broken rather than correct.

Part 4. Make it keep itself up to date

Step 9 filled in every gpa, and those numbers were right at the moment you ran it. Add one more score tomorrow and they are wrong again, because nothing re ran the function. Somebody has to remember, and somebody always forgets.

A **trigger** is the fix. It is a block of SQL the server runs by itself whenever a particular thing happens to a particular table. Nobody calls it and nobody can forget it. You attach it once and the server takes over.

Step 10. Create a trigger that recalculates the gpa

You want the gpa recalculated whenever test_scores changes. A trigger fires on one kind of change, so two of them cover adding a score and editing one. The statements are printed for you, the same as the function.

> **Command prompt again, for the same reason**
>
> These use DELIMITER too, so paste the block into student_practice.sql under the Step 10 heading and then run it at the command prompt, exactly as you did in Step 8.
>
> The same VS Code version works here as well: leave out the DELIMITER lines, drop the $$ after each END, and run the two CREATE TRIGGER statements one at a time.

```
-- $$ again, because a trigger body has semicolons in it too
DELIMITER $$

-- fires after a row is ADDED to test_scores
CREATE TRIGGER trg_scores_after_insert
AFTER INSERT ON test_scores
-- once for every row the statement touched, not once per statement
FOR EACH ROW
BEGIN
    -- NEW is the row that was just inserted, so NEW.student_id is
    -- whichever student that score belongs to
    UPDATE students
    SET gpa = calculate_gpa(NEW.student_id)
    WHERE student_id = NEW.student_id;
END$$

-- and the same again for a score being CHANGED
CREATE TRIGGER trg_scores_after_update
AFTER UPDATE ON test_scores
FOR EACH ROW
BEGIN
    UPDATE students
    SET gpa = calculate_gpa(NEW.student_id)
    WHERE student_id = NEW.student_id;
END$$

DELIMITER ;
```

> **What AFTER, FOR EACH ROW and NEW mean**
>
> **AFTER INSERT ON test_scores** is the event. The server runs this block once the insert has already succeeded, which matters here because the function has to be able to see the new score when it averages.
>
> **FOR EACH ROW** means it runs once per affected row. Insert five scores in one statement and the trigger runs five times, so five students get recalculated.
>
> **NEW** is the row being written, and you read its columns with a dot. NEW.student_id is how the trigger knows which student to update without you telling it.

Check both are registered before you test them:

```
show triggers;
```

Now prove it works. Do **one** of these two, and write down the gpa before and after.

Either add a score. Insert a quiz4 row for student 1, Jenna Wilson, with a score of 100. You write this one yourself, the same as Step 6.

Jenna's gpa before	Jenna's gpa after	Why
85.00	90.00	Her scores were 90 and 80, averaging 85. Adding 100 makes it 90 and 80 and 100, which averages 90

Or change a score. Update student 2, Robert Reese, so his quiz1 score becomes 100 instead of 85.

Robert's gpa before	Robert's gpa after	Why
84.00	89.00	His scores were 85, 72 and 95, averaging 84. Changing the 85 to 100 makes the average 89

Then select from students and check the number moved. You never called calculate_gpa and you never wrote an UPDATE against students. The trigger did both, the moment the score changed.

Select from **student_scores** as well. The view shows the new or changed score straight away, because a view re runs its query every time you ask, so there is nothing to keep up to date there.

> **What these two triggers do not cover**
>
> Delete a score and neither trigger fires, so the gpa is left standing at a number that is now wrong. Try it if you like: delete the quiz4 row you just added and watch Jenna stay on 90.00 when she should be back to 85.00.
>
> A third trigger on AFTER DELETE would close that gap. It would look the same except that a deleted row has no NEW, only **OLD**, so it would use OLD.student_id. You do not have to write it, but you should be able to say why it is needed.

Part 5. Push to GitHub

Step 11. Create the repository and submit the URL

Save student_practice.sql first. If you are not signed in to GitHub, click **Accounts** at the bottom of the left bar, choose **Sign in with GitHub**, and click **Authorize** in the browser.

1. Click **Source Control** in the left bar.
2. Click **Publish to GitHub**.

3. Name it **cis4720-lab5d**. It has to be a name you have not used.

4. Choose **Publish to a public repository**. A private one gives your instructor a 404 page instead of your work.

5. Tick **student_practice.sql**. Leave the **.vscode** folder unticked, because that is the one holding your password.

6. Go to github.com, open the repository, click the file and read it on the page. What is there is exactly what gets graded.

7. Open a private or incognito window and paste the URL in. A **404** means the repository is private, so go to **Settings** and change the visibility to public.

8. Paste the URL into the Lab 5d submission box on Canvas.

> **Sign out of GitHub before you leave the lab**
>
> This is a shared PC and your sign in stays on it until somebody removes it. Click **Accounts**, then your username, then **Sign Out**.

How it is graded

Steps	What is being checked	Points
Setup	Connected, cis_students_db created, working in the right schema	1.0
1	students created with AUTO_INCREMENT on the primary key, and described	1.5
2 to 3	Six rows inserted both ways, with last_insert_id run after each	1.0
4	James Toor deleted and the table selected again	0.5
5	test_scores created with the foreign key pointing at students(student_id)	1.5
6	All ten score rows inserted	0.5
7	student_scores created from the outer join, with the names joined up, and queried both ways	1.5
8 to 9	The function created, shown, and run so every gpa is filled in	0.5
10	Both triggers created, and one of them shown working on a real change	1.0
All	Required. A comment in your own words above every statement	1.0
11	A new public repository with the file in it, URL submitted	0.0

Total 10 points. The due date is listed on Canvas LMS.

If something goes wrong

What you see	What it means
ERROR 1064, syntax error near 'DELIMITER $$' at line 1	You ran Step 8 or Step 10 in VS Code. DELIMITER is a command prompt instruction, not SQL, so the server rejects it. Run those two steps at the command prompt, or use the version without the DELIMITER lines.
ERROR 1046, no database selected	You never ran the USE, or your tool forgot it. Run SELECT DATABASE() to see where you are.

What you see	What it means
Your tables are in the wrong schema	Same cause. Add a SQLTools connection pointing at cis_students_db and use that one.
ERROR 1005, errno 150, foreign key incorrectly formed	Step 5. You pointed the key at a column that does not exist. The column in students is student_id, not id.
ERROR 1146, table 'dbo.students' doesn't exist	dbo is SQL Server, not MySQL. Here the table is just students, or cis_students_db.students if you want to spell it out.
last_insert_id() returns 0	A new connection, which has nothing to remember. Run the insert and the select together.
last_insert_id() returns 1 after four rows	Correct. A multiple row insert reports the first id of the batch.
ERROR 1064 on the DECLARE line	Your editor split the function at the first semicolon inside it. Run it at the command prompt instead.
ERROR 1304, function already exists	You ran Step 8 twice. Run DROP FUNCTION calculate_gpa; first, then create it again.
ERROR 1418, this function has none of DETERMINISTIC...	Your copy is missing the DETERMINISTIC or READS SQL DATA line. Both are in the block in Step 8.
Every gpa comes out 0.00	Step 6 did not run, or the student_id values in test_scores do not match the ones in students.